

CS-334: Machine Learning

Lecture 06: Linear Regression

(with In-Class Exercise #5)

09/15/2026

Prof. Shengpu Tang



Announcements

HW1 Due this past Sun 11:59pm

Topics covered (Prereqs, Lec 1&2)

- Review of vectors & planes
- Review of linear algebra
- numpy and pandas
- Data exploration
- Plotting in python

Solutions: tomorrow at 11:59pm

Grades: early next week

HW2 Released; in two weeks Sun 11:59pm

Topics covered

- Decision Boundaries (Lec #3)
- Perceptron (Lec #4)
- GD, SGD (Lec #5)
- Linear Regression (Lec #6)
- Regularization (Lec #7)

Extra Credit, due 3 days after HW2 deadline

- Weighted Linear Regression

Announcements

- Lecture slides and notes
- Textbook readings (optional)
- Panopto recording

Recap

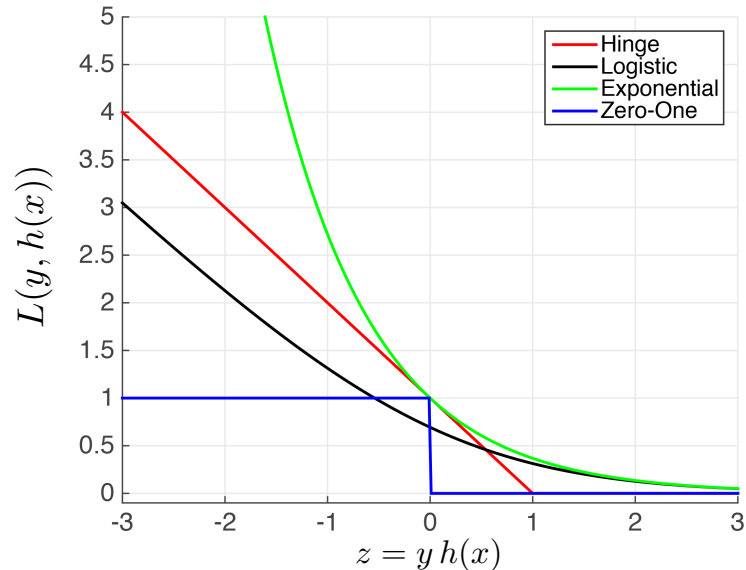
Empirical Risk

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \text{loss}(y^{(i)}, \vec{\theta} \cdot \vec{x}^{(i)})$$

Recap

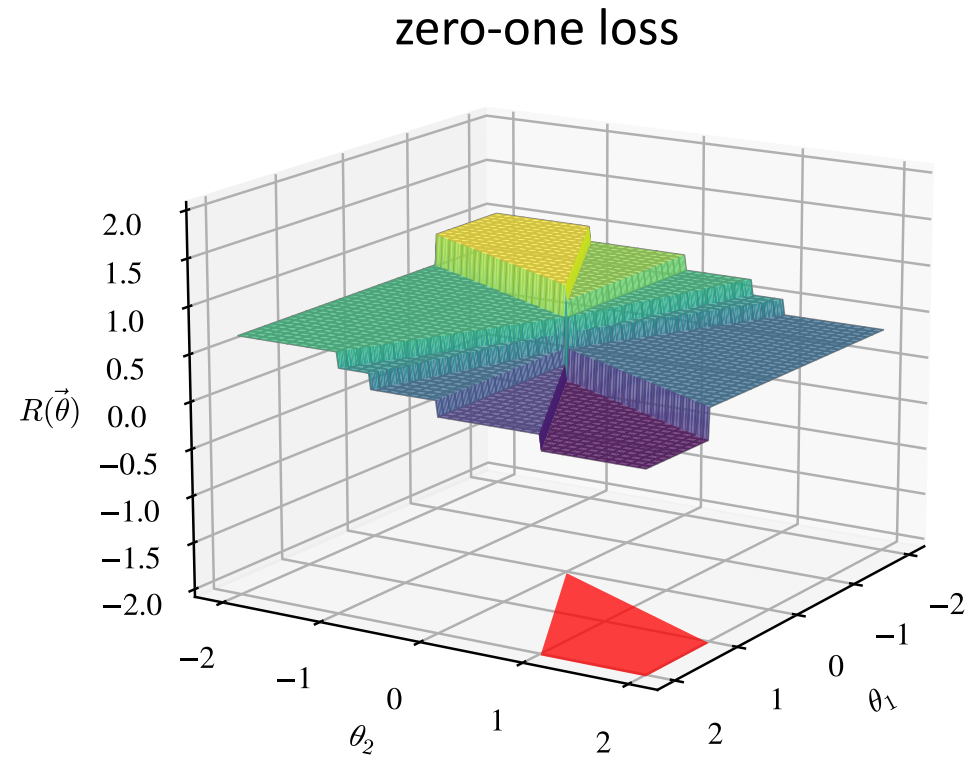
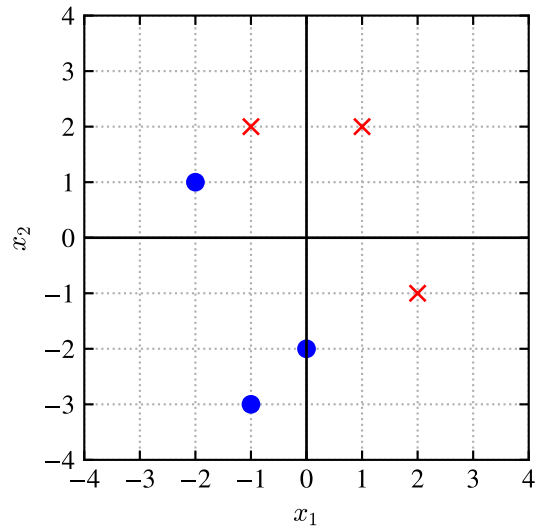
Empirical Risk based on Hinge Loss

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \max\{0, 1 - y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}\}$$

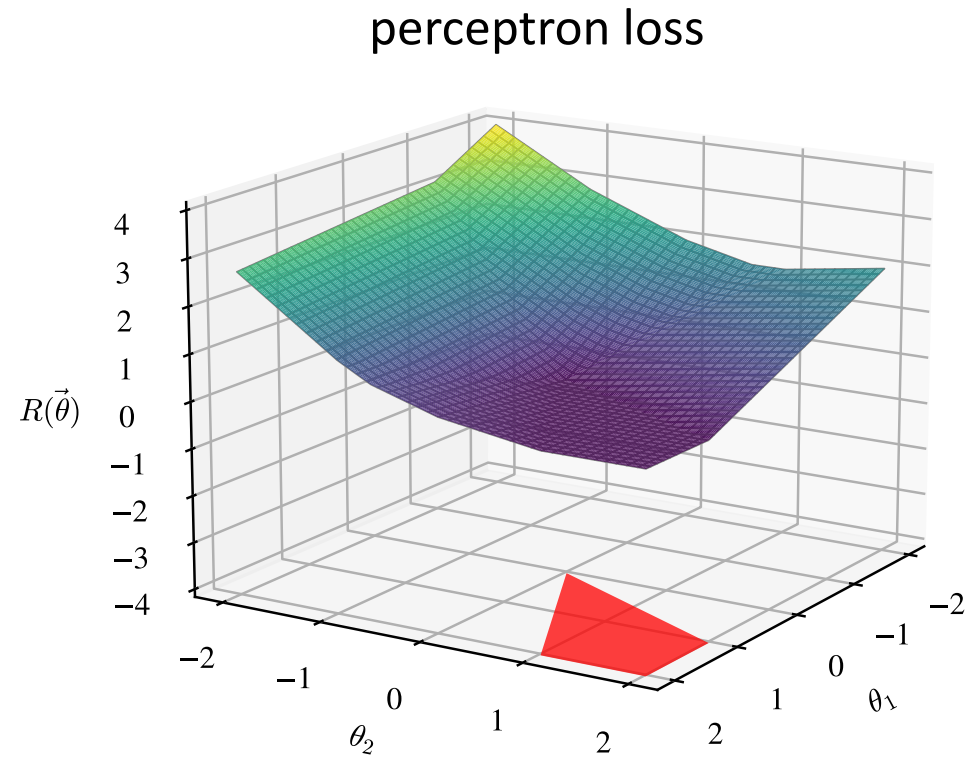
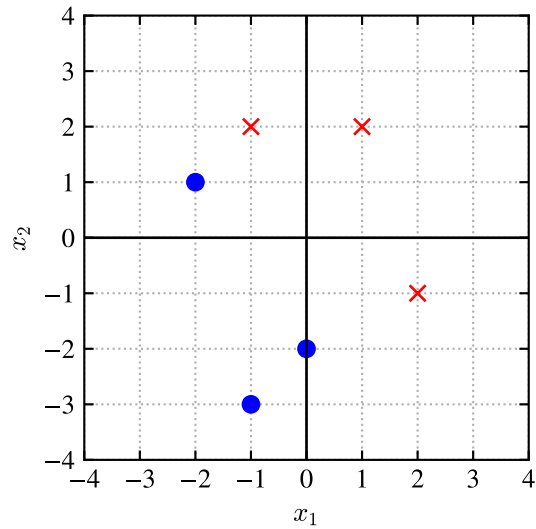


Why did we consider hinge loss instead of zero-one loss?

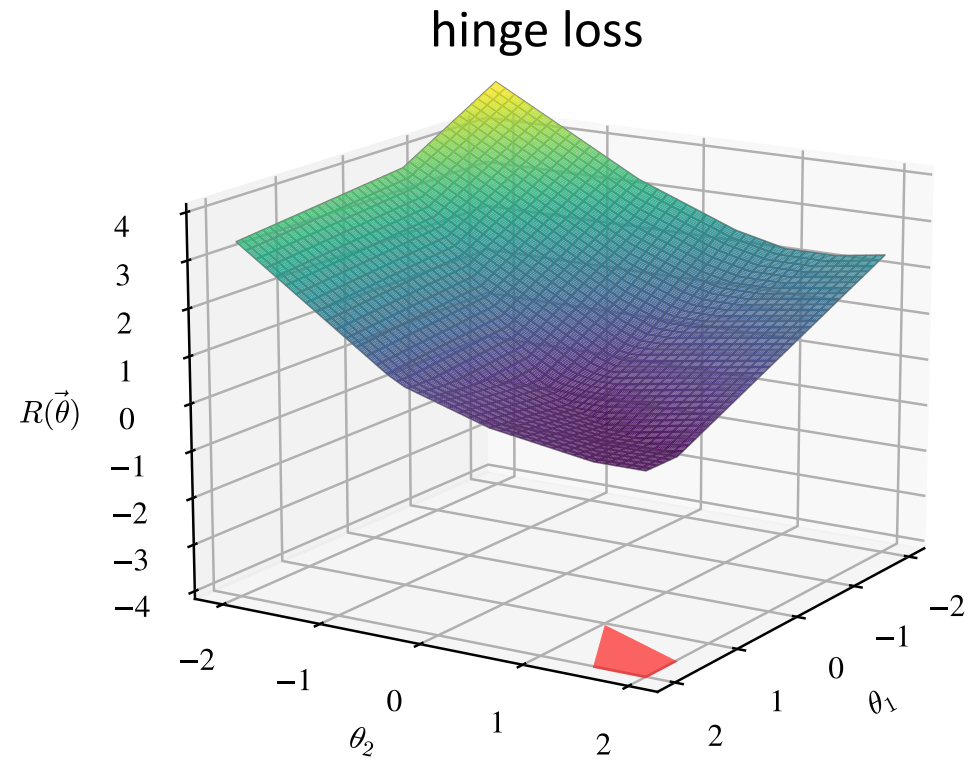
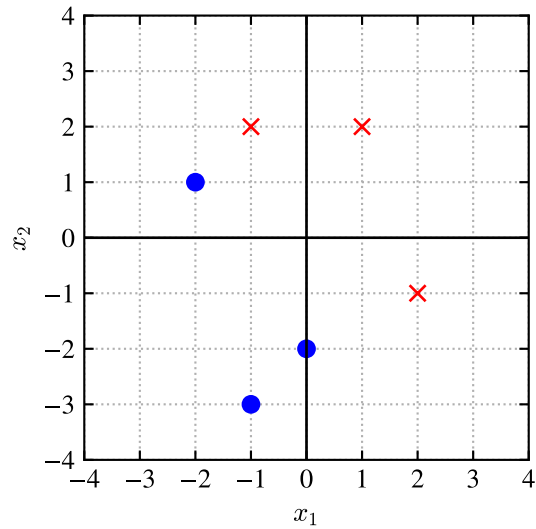
Loss Surface



Loss Surface



Loss Surface



Gradient

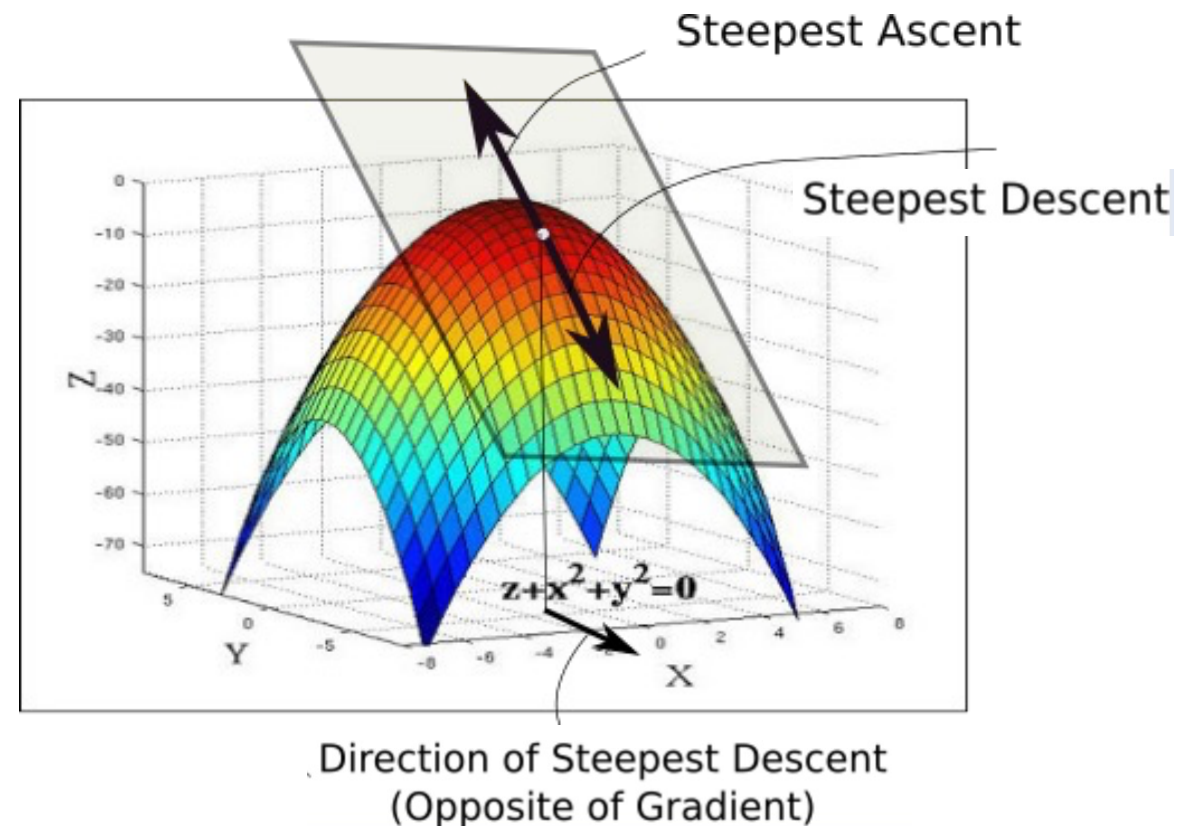
$f(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}$ is some function

We want to find a $\vec{\theta}^*$ that minimizes $f(\vec{\theta})$

Gradient captures the “shape” of the function

$$\nabla_{\vec{\theta}} f(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}^d$$

for every $\vec{\theta}$, it tells us which way to go such that $f(\vec{\theta})$ increases the fastest

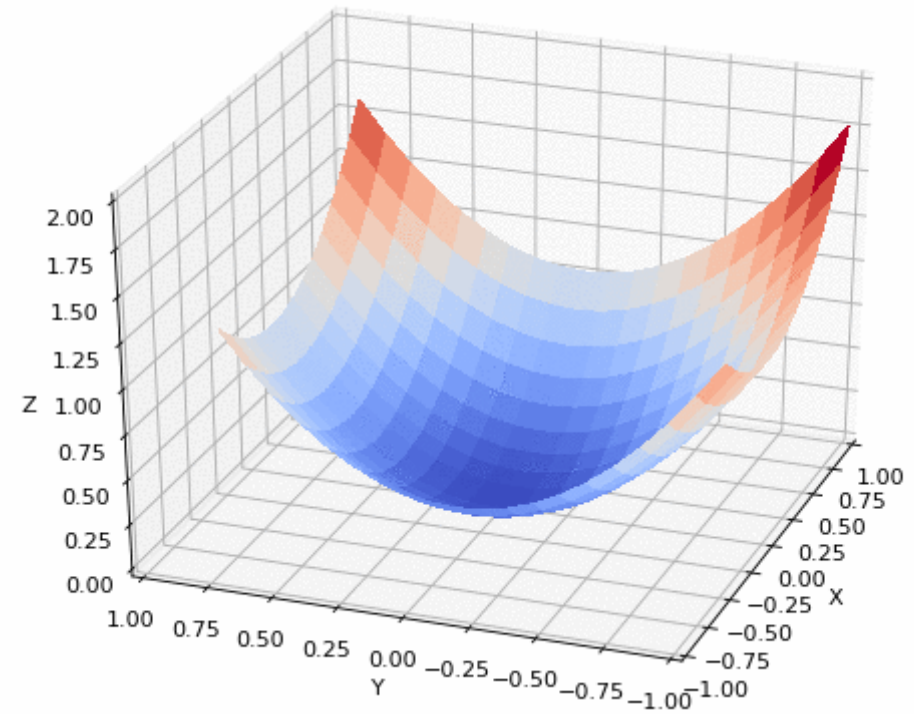


Gradient Descent

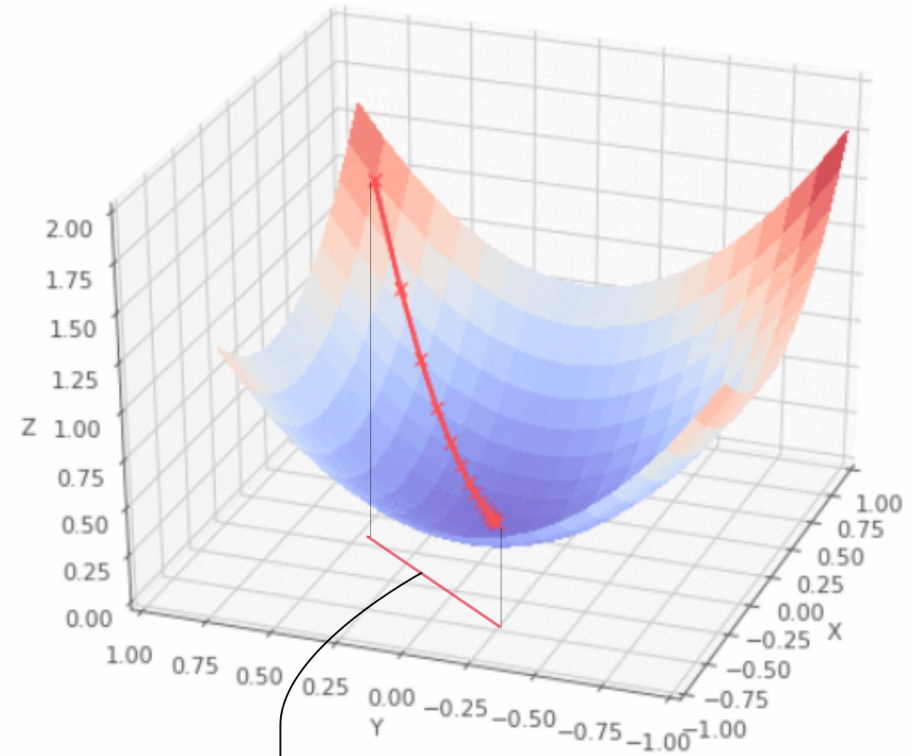
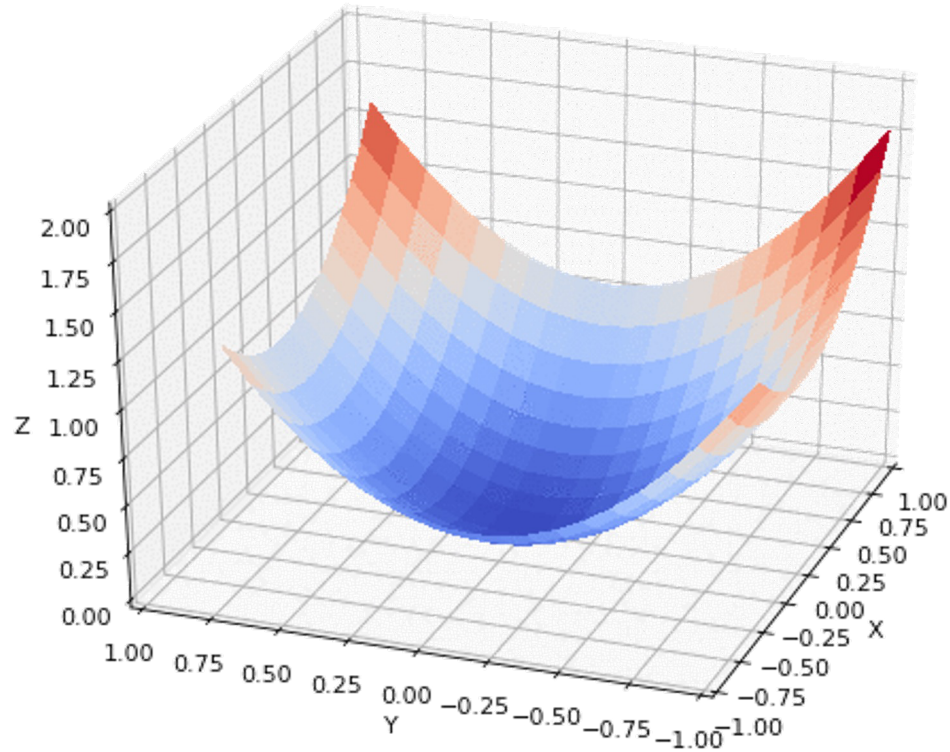
$f(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}$ is some function

We want to find a $\vec{\theta}^*$ that minimizes $f(\vec{\theta})$

- Start somewhere
- Take a small step in the opposite direction of the gradient
- Repeat



GD trajectory



Real Trajectory of G.D.

Recap: (Batch) Gradient Descent

Goal: minimize empirical risk $R_N(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}$

$$k = 0, \vec{\theta}^{(0)} = \vec{0}$$

while not converged

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k \nabla_{\vec{\theta}} R_N(\vec{\theta})|_{\vec{\theta}=\vec{\theta}^{(k)}}$$

$$k = k + 1$$

Recap: Stochastic Gradient Descent

Goal: minimize empirical risk $R_N(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}$

$$k = 0, \vec{\theta}^{(0)} = \vec{0}$$

while not converged

 randomly shuffle points

 for $i = 1 \dots N$

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k \nabla_{\vec{\theta}} \text{loss}(y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)})|_{\vec{\theta}=\vec{\theta}^{(k)}}$$

$$k = k + 1$$

GD vs SGD

Goal: minimize empirical risk $R_N(\vec{\theta}) : \mathbb{R}^d \rightarrow \mathbb{R}$

$k = 0, \vec{\theta}^{(0)} = \vec{0}$

while not converged

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k \nabla_{\vec{\theta}} R_N(\vec{\theta})|_{\vec{\theta}=\vec{\theta}^{(k)}}$$

$k = k + 1$

How to set
learning rate / step size?

Details in Lec #5 notes.

$k = 0, \vec{\theta}^{(0)} = \vec{0}$

while not converged

randomly shuffle points

for $i = 1 \dots N$

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta_k \nabla_{\vec{\theta}} \text{loss}(y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)})|_{\vec{\theta}=\vec{\theta}^{(k)}}$$

$k = k + 1$

What's the
update rule?

How do we know
when the algorithm
has converged?

Convergence Criteria

Which of the following are reasonable convergence criteria?

- (a) Check if training error is zero, same as in perceptron.
- (b) Check if empirical risk is zero.
- (c) Check if empirical risk stops changing.
- (d) Check if parameter vector stops changing.

Update Rule

Objective: minimize empirical risk with hinge loss

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \max\{0, 1 - y^{(i)} \vec{\theta} \cdot \vec{x}^{(i)}\}$$

Stochastic Gradient Descent

if $y^{(i)} \vec{\theta}^{(k)} \cdot \vec{x}^{(i)} \geq 1$

no update

else

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + \eta_k y^{(i)} \vec{x}^{(i)}$$

Gradient Descent

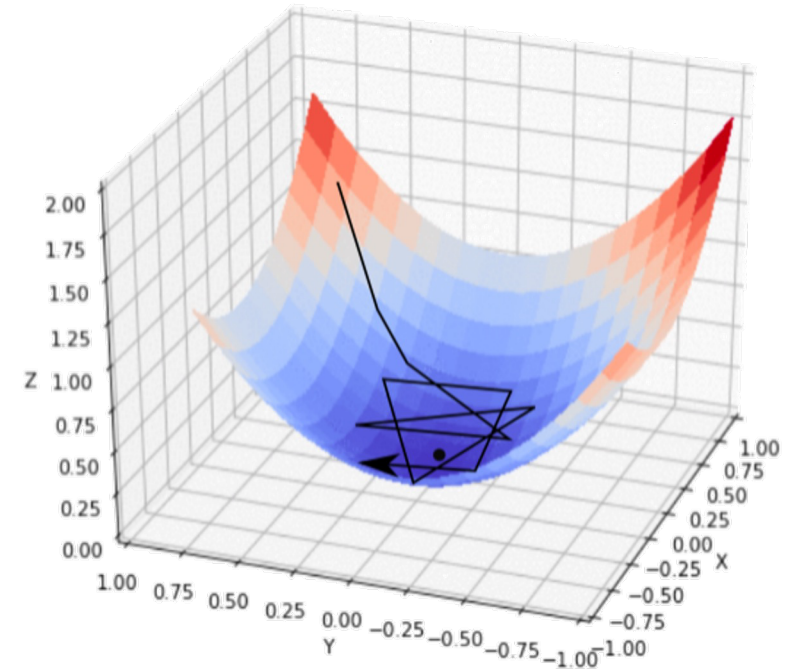
$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)}$$

$$+ \eta_k \frac{1}{N} \sum_{i=1}^N \mathbb{1}[y^{(i)} \vec{\theta}^{(k)} \cdot \vec{x}^{(i)} < 1] (y^{(i)} \vec{x}^{(i)})$$

Learning Rate / Step Size

What happens if the learning rate is too large?

What happens if the learning rate is too small?

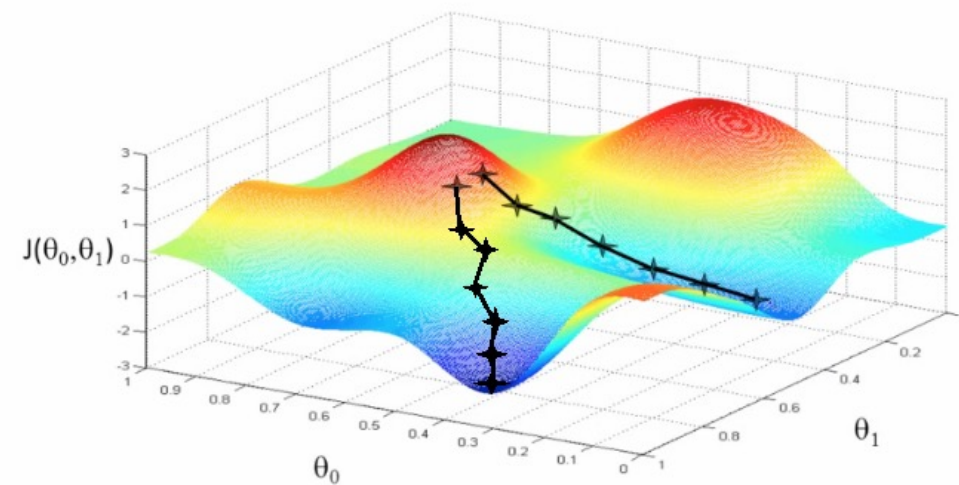
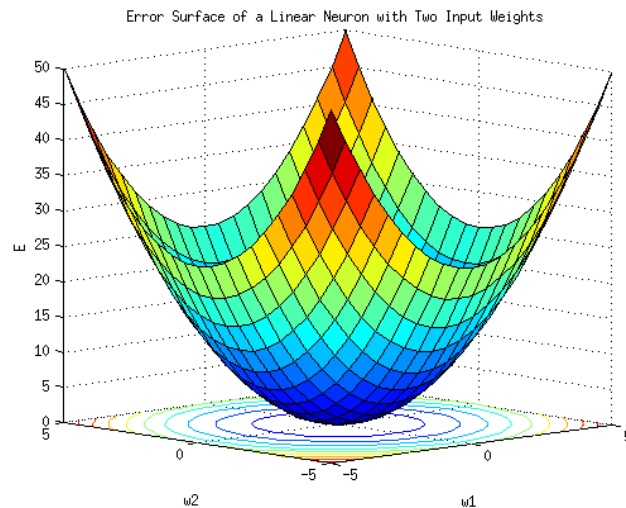


A “hyperparameter” that can (and should) be tuned.

Constant learning rate vs Decaying learning rate $\eta_k = \frac{1}{(1 + k)}$

GD & SGD: Convergence Theorems

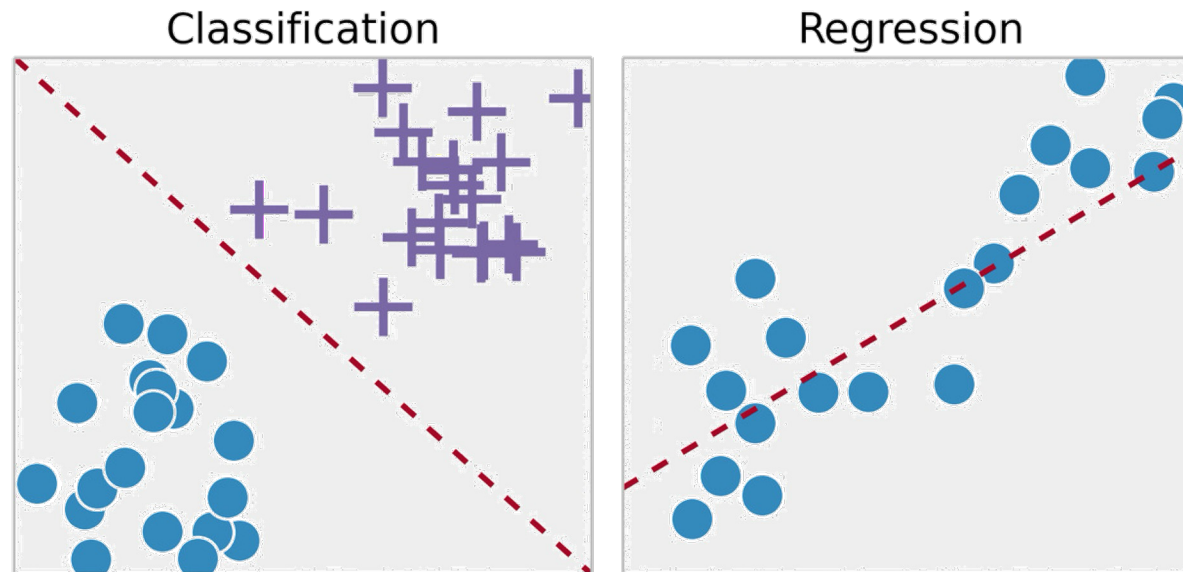
- With an appropriate learning rate, if the objective function is convex, (S)GD will converge to the global minimum *almost surely*.
- (S)GD is a general optimization algorithm that can also be applied to non-convex objective functions -- it converges to a local minimum.



Regression

Classification task: $y \in \{-1, +1\}$ or $y \in \{0, 1\}$

Regression task: $y \in \mathbb{R}$



Regression

Given **training data** containing a set of **labeled examples**,
come up with a **rule** that maps an **unlabeled example** to the **correct label**.

Given $D = \{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^N$ $\vec{x}^{(i)} \in \mathbb{R}^d$ $y \in \mathbb{R}$

Find a “good” $f : \mathbb{R}^d \rightarrow \mathbb{R}$

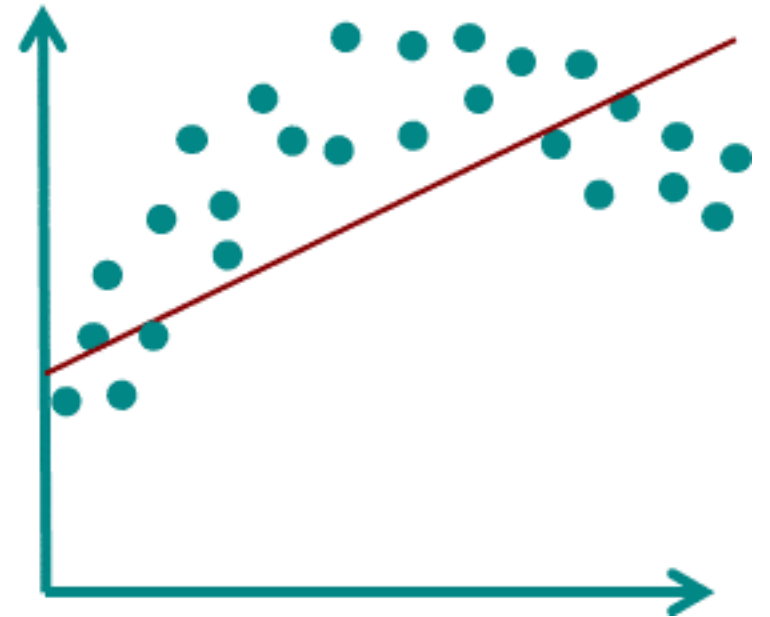
Linear Regression

Goal - learn a linear function of the feature vector:

$$f(\vec{x}; \vec{\theta}, b) = \vec{\theta} \cdot \vec{x} + b$$

Questions:

- How do we measure “error”? What criterion should we use to judge if a regression model is “good”?
- What algorithms can we use to optimize this criterion?



Lecture 6. Linear Regression

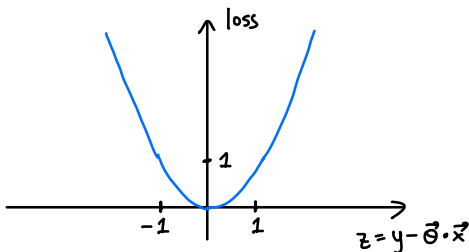
Regression

- Empirical Risk (proxy for training error)

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \text{loss}(y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)})$$

- Squared loss: $\text{loss}(z) = \frac{z^2}{2}$

aka "ordinary least squares"
errors are measured parallel to y-axis.



squared loss is
✓ continuous
✓ differentiable
✓ convex.

Intuition: permit small discrepancies
but penalize large deviations.

What loss functions
have we seen?

- zero-one loss (Lec 3)
- perceptron loss (Lec 4)
- hinge loss (Lec 5)
- logistic loss (Lec 9)

$z = y^{(i)}(\vec{\theta} \cdot \vec{x})$
for classification

Goal: minimize empirical risk with squared loss

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \frac{(y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)})^2}{2}$$

What algorithm can we use to optimize $R_N(\vec{\theta})$?
that we've already seen

SGD!

Let's derive the update rule.

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} - \eta \nabla_{\vec{\theta}} \text{loss}(y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)}) \big|_{\vec{\theta} = \vec{\theta}^{(k)}}$$

$$\begin{aligned} & \nabla_{\vec{\theta}} \frac{(y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)})^2}{2} \\ &= \frac{1}{2} \times 2 (y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)}) \times \nabla_{\vec{\theta}} (y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)}) \\ &= (y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)}) (-\vec{x}^{(i)}) \end{aligned}$$

$$\Rightarrow \vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + \eta (y^{(i)} - \vec{\theta}^{(k)} \cdot \vec{x}^{(i)}) \vec{x}^{(i)}$$

Calc Refresher

Chain rule: $\frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$

Example:

$$\frac{d}{dx} \frac{(y - wx)^2}{2} = (y - wx) \cdot (-w)$$

$$\left(\begin{array}{ll} f(z) = \frac{z^2}{2} & f'(z) = z \\ g(x) = y - wx & g'(x) = -w \end{array} \right)$$

► Stochastic Gradient Descent for least squares regression

$$\vec{\theta}^{(1)} = \vec{\theta}, k=0$$

while convergence criteria not met
randomly shuffle points

for $i = 1 \dots N$

$$\vec{\theta}^{(k+1)} = \vec{\theta}^{(k)} + \eta_k (y^{(i)} - \vec{\theta}^{(k)} \cdot \vec{x}^{(i)}) \vec{x}^{(i)}$$

$k++$

Compared to hinge loss in classification:

- convergence criteria, learning rate considerations $\rightarrow$ same as before.
- Difference: here we make an update for every example unless error $= 0$.

► Closed-Form Solution

Since $R_N(\vec{\theta})$ with squared loss is a convex function and differentiable everywhere, we can try to minimize it directly by setting the gradient to zero and solving analytically.

Calc Refresher.

Let $f(x) = (ax + b)^2$. $f(x)$ is convex i.e. concave up.

Find the x^* that minimizes f . "critical point"

① find the first derivative

$$f'(x) = 2a(ax + b)$$

② set $f'(x)$ to 0 and solve for x

$$2a(ax + b) = 0 \Rightarrow x^* = -\frac{b}{a}$$

First, rewrite empirical risk in matrix notation (see HW1)

$$X = \begin{bmatrix} - & \vec{x}^{(1)T} & - \\ & \vdots & \\ - & \vec{x}^{(N)T} & - \end{bmatrix} \quad N \times d \text{ matrix} \quad \vec{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{bmatrix} \quad N \times 1 \text{ column vector}$$

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} (y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)})^2$$

$$= \frac{1}{N} \cdot \frac{1}{2} (X\vec{\theta} - \vec{y})^T (X\vec{\theta} - \vec{y}) \quad \text{or equivalently} = \frac{1}{N} \cdot \frac{1}{2} \|X\vec{\theta} - \vec{y}\|_2^2$$

$$= \frac{1}{N} \cdot \frac{1}{2} (\vec{\theta}^T X^T X \vec{\theta} - \vec{\theta}^T X^T \vec{y} - \vec{y}^T X \vec{\theta} + \vec{y}^T \vec{y})$$

$$= \frac{1}{N} \cdot \frac{1}{2} (\vec{\theta}^T (X^T X) \vec{\theta} - 2 \vec{\theta}^T (X^T \vec{y}) + \vec{y}^T \vec{y})$$

Step 1: find the gradient.

$$\left. \begin{aligned} \nabla_{\vec{\theta}} (\vec{\theta}^T A \vec{\theta}) &= (A + A^T) \vec{\theta} \\ \nabla_{\vec{\theta}} (\vec{\theta}^T \vec{v}) &= \nabla_{\vec{\theta}} (\vec{v}^T \vec{\theta}) = \vec{v} \end{aligned} \right\} \Rightarrow \nabla_{\vec{\theta}} R_N(\vec{\theta}) = \frac{1}{N} ((X^T X) \vec{\theta} - (X^T \vec{y}))$$

Step 2: set gradient to zero and solve.

$$\nabla_{\vec{\theta}} R_N(\vec{\theta}) \big|_{\vec{\theta} = \vec{\theta}^*} = \vec{0}$$

$$(X^T X) \vec{\theta}^* - X^T \vec{y} = \vec{0}$$

$$(X^T X) \vec{\theta}^* = X^T \vec{y}$$

$$\vec{\theta}^* = (X^T X)^{-1} X^T \vec{y}$$

Practical Notes:

- OLS solution $\vec{\theta}^* = (X^T X)^{-1} X^T \vec{y}$ when $X^T X$ is invertible.
computational complexity of inverting $X^T X$ is $O(d^3)$
($d \times d$ matrix)
- Invertibility of $X^T X$ "Gram matrix"
 - Why might it be non-invertible / singular / degenerate / rank-deficient?
Recall $\text{rank}(X^T X) \leq \text{rank}(X) \leq \min(N, d)$ (see HW1)
when d is large, there may be redundant features
such that columns of X are not linearly independent. "multicollinearity"
e.g. if $d > N$, $X^T X$ is a $d \times d$ matrix with rank at most $N \Rightarrow$ not invertible
 $\Rightarrow \vec{\theta}^*$ solution is not unique.
e.g. duplicated feature $x_1 = x_2$
then if $[\theta_1, \theta_2]$ is a solution, $[\theta_1 + c, \theta_2 - c]$ is also a solution.
 - What to do if $X^T X$ is not invertible?
 - use Moore-Penrose pseudo inverse
 $\vec{\theta}^* = (X^T X)^+ X^T \vec{y} = X^+ \vec{y}$
(use `scipy.linalg.pinv` in HW2)
 A^+ is equivalent to A^{-1} if A invertible
otherwise gives the "minimum-norm" solution; if $x_1 = x_2$, then $\theta_1 = \theta_2$
duplicated feature weight split equally.
 - Regularization (next lecture)

Appendix: common differentiation rules

derivatives		gradients	
$f(x)$	$\frac{d}{dx} f(x)$	$f(\vec{x})$	$\nabla_{\vec{x}} f(\vec{x})$
ax	a	$\vec{v}^T \vec{x}$	$\vec{v}$
x^2	$2x$	$\vec{x}^T \vec{x}$	$2\vec{x}$
ax^2	$2ax$	$\vec{x}^T A \vec{x}$	$(A + A^T) \vec{x}$
		if A symmetric	$= 2A\vec{x}$

Appendix: closed-form solution in alternative notation.

$$R_N(\vec{\theta}) = \frac{1}{N} \sum_{i=1}^N \frac{(y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)})^2}{2}$$

Step 1: find the gradient.

$$\begin{aligned}
 \nabla_{\vec{\theta}} R_N(\vec{\theta}) &= \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \vec{\theta} \cdot \vec{x}^{(i)}) (-\vec{x}^{(i)}) \quad \leftarrow \text{what we've derived in SGD for squared loss.} \\
 &= \frac{1}{N} \left(- \sum_{i=1}^N y^{(i)} \vec{x}^{(i)} + \sum_{i=1}^N (\vec{\theta} \cdot \vec{x}^{(i)}) \vec{x}^{(i)} \right) \\
 &= \frac{1}{N} \left(- \sum_{i=1}^N y^{(i)} \vec{x}^{(i)} + \sum_{i=1}^N \underbrace{\vec{x}^{(i)} \vec{x}^{(i)T}}_{d \times d} \vec{\theta} \right) \quad \text{rearranged so that we can factor out } \vec{\theta}. \\
 &= \frac{1}{N} \left(- \underbrace{\left(\sum_{i=1}^N y^{(i)} \vec{x}^{(i)} \right)}_{d \times 1} + \underbrace{\left(\sum_{i=1}^N \vec{x}^{(i)} \vec{x}^{(i)T} \right)}_{d \times d} \underbrace{\vec{\theta}}_{d \times 1} \right) \quad (\vec{\theta} \cdot \vec{x}^{(i)}) \vec{x}^{(i)} = \vec{x}^{(i)} (\vec{x}^{(i)} \cdot \vec{\theta}) = \vec{x}^{(i)} \vec{x}^{(i)T} \vec{\theta}. \text{ scalar} \times \text{vector}
 \end{aligned}$$

Step 2: set gradient to zero.

$$\begin{aligned}
 \nabla_{\vec{\theta}} R_N(\vec{\theta})|_{\vec{\theta}=\vec{\theta}^*} &= \vec{0} \\
 - \underbrace{\left(\sum_{i=1}^N y^{(i)} \vec{x}^{(i)} \right)}_{X^T \vec{y}} + \underbrace{\left(\sum_{i=1}^N \vec{x}^{(i)} \vec{x}^{(i)T} \right)}_{X^T X} \vec{\theta}^* &= \vec{0} \quad \Rightarrow \quad \begin{aligned} 0 &= -X^T \vec{y} + X^T X \vec{\theta}^* \\ X^T X \vec{\theta}^* &= X^T \vec{y} \\ \vec{\theta}^* &= (X^T X)^{-1} X^T \vec{y} \end{aligned}
 \end{aligned}$$

Recall: $X = [\vec{x}^{(1)}, \vec{x}^{(2)}, \dots, \vec{x}^{(N)}]^T$ ($N \times d$ matrix)

$\vec{y} = [y^{(1)}, y^{(2)}, \dots, y^{(N)}]^T$ ($N \times 1$ column vector)

$$X^T \vec{y} = \begin{bmatrix} | & | & | \\ \vec{x}^{(1)} & \vec{x}^{(2)} & \dots & \vec{x}^{(N)} \\ | & | & | \end{bmatrix} \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix}$$

$d \times N$ matrix $N \times 1$

$$= \vec{x}^{(1)} y^{(1)} + \vec{x}^{(2)} y^{(2)} + \dots + \vec{x}^{(N)} y^{(N)}$$

$$= \sum_{i=1}^N y^{(i)} \vec{x}^{(i)}$$

$$X^T X = \begin{bmatrix} | & | & | \\ \vec{x}^{(1)} & \vec{x}^{(2)} & \dots & \vec{x}^{(N)} \\ | & | & | \end{bmatrix} \begin{bmatrix} - \vec{x}^{(1)T} - \\ - \vec{x}^{(2)T} - \\ \vdots \\ - \vec{x}^{(N)T} - \end{bmatrix}$$

$d \times N$ matrix $N \times d$ matrix

$$= \vec{x}^{(1)} \vec{x}^{(1)T} + \vec{x}^{(2)} \vec{x}^{(2)T} + \dots + \vec{x}^{(N)} \vec{x}^{(N)T}$$

$$= \sum_{i=1}^N \vec{x}^{(i)} \vec{x}^{(i)T}$$